At N=96, depth cannot buy back MLP width.

Adding four layers to the thin model made it WORSE.

Narrowing dim_feedforward 1024 -> 256 at N=96 costs 7.1 points (0.8407 vs 0.9113, p=2.3e-11). Going from there to L=12 costs another 3.1, landing at 0.8100 (vs the wide knee, -0.1013, p=8.4e-19).

THE PREDICTION THIS KILLS. At N=48 the thin knee sat 1.5x above the wide one (L*=4 wide, 6 thin, finding 14). I predicted the same factor at N=96, so L*(96, ff=256) ~ 12. Instead L=12/ff=256 clears NONE of the six rho=0.60 depth bins (L=8/ff=256 clears three), so L*(96, ff=256) > 12: the width cost is SUPER-multiplicative in N, which no cell in this project had shown.

AND IT IS NOT A PARAMETER STORY. L=12/ff=256 has 5.01M parameters against the wide knee's 6.58M -- 0.76x -- but L=8/ff=256 has only 3.43M and beats it by 3.1 points. Within the thin column, +1.58M parameters spent on depth bought a LOSS. Finding 13's 'depth is the cheaper currency' does not survive to N=96.

job 49816217, COMPLETED 15:50:15, ExitCode 0:0 | commit 3873723 | scripts/slurm/sudoku2_n96_ff256_lstar.sbatch | seed 0 | shuffle_adj=False, so every number here is a DECODING score

What the model actually sees

Every token is one categorical id into a 106-symbol vocabulary (N=96, q=4, degree=5). No text, no digits-as-characters.

VOCABULARY

0-5	BEGIN_HINTS BEGIN_GRAPH BEGIN_REASONING BEGIN_ROUND SOLVED UNRESOLVED	*section + round markers*
6-9	COLOR_0 COLOR_1 COLOR_2 COLOR_3	*the q colours*
10-105	N0 N1 ... N95	*one id per node, no structure in the id*

GRAMMAR

BEGIN_HINTS	(NODE COLOR)*
BEGIN_GRAPH	(NODE (NODE){degree})*
BEGIN_REASONING	(BEGIN_ROUND (NODE COLOR)+)*
SOLVED	

The graph section is a neighbour table, not an image: node, then its degree neighbours. THE ORDER WITHIN A ROW IS THE LEAK of finding 16.

THIS INSTANCE, 783 TOKENS (seed 50000000026)

HINTS	BEGIN_HINTS N4 COLOR_1 N6 COLOR_2 N10 COLOR_3 N13 COLOR_1 N14 COLOR_0 N15 COLOR_2 ... (99 tokens)
GRAPH	BEGIN_GRAPH N0 N57 N89 N27 N39 N47 N1 N51 N33 N89 N76 N6 ... (577 tokens)
REASONING	BEGIN_REASONING BEGIN_ROUND N1 COLOR_3 N2 COLOR_0 N3 COLOR_2 N7 COLOR_1 N8 COLOR_2 N9 ... (107 tokens)

Loss is on the REASONING section only: 11 rounds, 47 deductions, 94 target tokens. Everything before BEGIN_REASONING is the problem statement.

One real reasoning trace, at N=96

49 hints of 96 nodes, then 11 rounds of simultaneous deduction. Hints: N4=1 N6=2 N10=3 N13=1 N14=0 N15=2 N17=1 N18=2 ...

THE TRACE, AS EMITTED

```
BEGIN_ROUND          N1 COLOR_3 N2 COLOR_0 N3 COLOR_2 N7 COLOR_1 ...
  20 deduced
BEGIN_ROUND          N30 COLOR_3 N35 COLOR_3 N37 COLOR_0 N48 COLOR_0 ...
  9 deduced
BEGIN_ROUND          N0 COLOR_2 N32 COLOR_1 N44 COLOR_3 N45 COLOR_0 ...
  5 deduced
BEGIN_ROUND          N77 COLOR_0 N79 COLOR_1
  2 deduced
... 7 further rounds

SOLVED
```

WHY ONE DEDUCTION FIRES

node N1, neighbours [51, 33, 89, 76, 6]

colours known before this round:

```
   [0, 1, 2]  (3 = q-1)
```

only colour left: 3

```
so the trace emits  N1 COLOR_3
```

The model is never shown this reasoning -- only the node/colour pair.

A round deduces every node whose neighbours already cover q-1 colours, all at once. Reasoning depth T is the number of rounds: a deduction in round 3 depends on ones made in rounds 1 and 2, which is the serial computation that network depth has to supply.

shuffle_adj=False here, so each node's neighbour row is still sorted by the neighbour's TRUE colour -- finding 16's leak. That is deliberate: this run extends the leaked-order L* ladder rather than starting a second one.

T=11 rounds, the 90th percentile of 40 draws from this config (range 1-19); seed 50000000026, shuffle_adj=False, shuffle_events=False; evidence from sudoku_tokens.evidence_rounds (pre-round state, as DD saw it)

The grid this job was filling in: L* vs (N, ff)

L* is the smallest num_layers reaching 0.90 on depth-T instances at fixed realized density. Before this run, ff=256 was measured at one size only.

	L* at ff=1024	L* at ff=256	width factor
N=24	3	not measured	--
N=48	4	6	1.5x (finding 14)
N=96	8	> 12	> 1.5x (THIS RUN)

WHAT 'ff' IS

dim_feedforward: the hidden width of the per-layer MLP. At d_model=256, ff=1024 is the usual 4x expansion and ff=256 is 1x -- no expansion at all. Attention, d_model, heads, lr, batch and epochs are untouched.

cell	n_params	vs wide knee	val_nll	pool solve_rate	rho=0.60 bins >= 0.90
L=8, ff=1024	6,578,026	1.00x	1.4141	0.9113	6 of 6
L=8, ff=256	3,426,154	0.52x	1.4337	0.8407	3 of 6
L=12, ff=256	5,009,258	0.76x	1.4108	0.8100	0 of 6

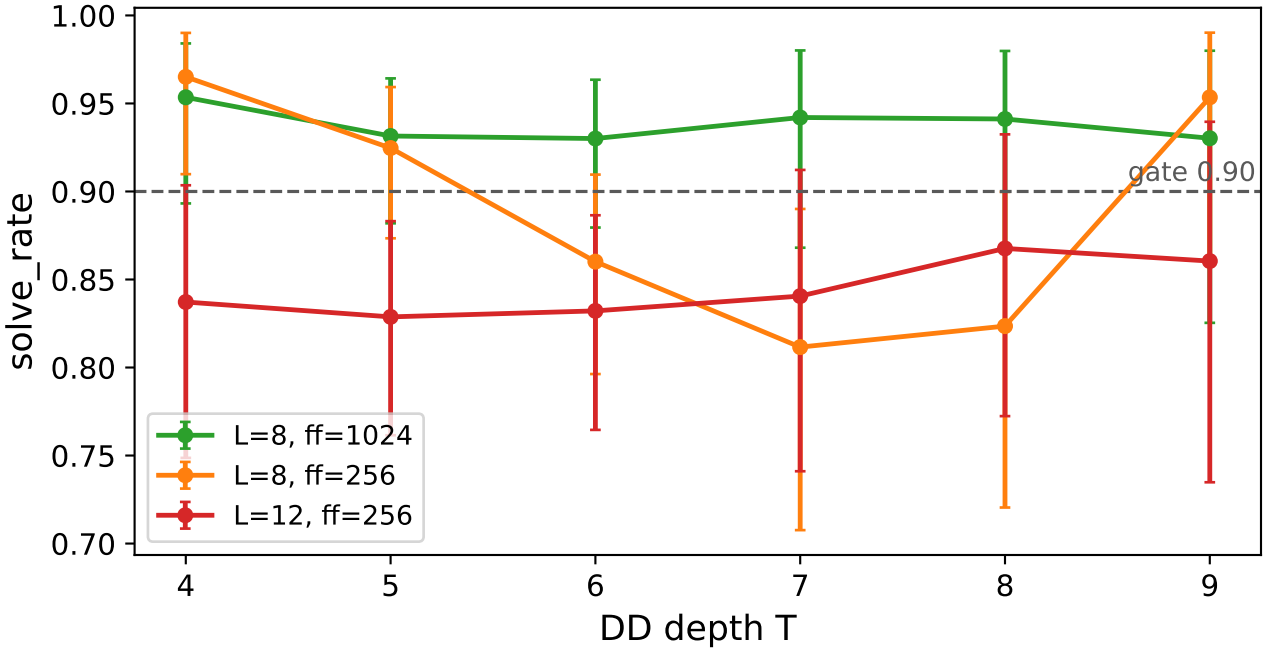
Read the last two columns together. The pool rate falls monotonically down the table, and so does the bin count -- but the parameter count does NOT: the worst cell has 1.46x the parameters of the middle one. Whatever the thin MLP costs, it is not capacity in the counting sense, and more layers do not substitute for it.

And val_nll ranks the three cells in almost the REVERSE order: the worst cell on the pool (0.8100) has the BEST teacher-forced nll (1.4108). Next-token loss and free generation come apart here, which is the first thing the mechanism page explains.

L* at N=24 and N=48 from findings 18 and 14; wide knee at N=96 from finding 22 (job 49639312); all at shuffle_adj=False

Six depth bins, same instances, three columns

rho=0.60, realized density 0.594 in every bin. The depth eval used the same --n-pool 600 --min-per-bin 40 and generator stream as finding 22, so the bins come out identical row by row and the columns compare bin by bin.



	T=4	T=5	T=6	T=7	T=8	T=9	CA p
L=8, ff=1024	0.953	0.932	0.930	0.942	0.941	0.930	0.814
L=8, ff=256	0.965	0.925	0.860	0.812	0.824	0.953	0.026
L=12, ff=256	0.837	0.829	0.832	0.841	0.868	0.860	0.503

n per bin: T=4:86, T=5:146, T=6:143, T=7:69, T=8:68, T=9:43

THE T=9 BIN IS NOT A RECOVERY

Both thin cells jump at T=9 (0.953 and 0.860). That bin has n=43, the smallest in the sweep, and the same non-monotone bounce appears in the wide column of finding 22. Deep instances that survive the min-per-bin filter are selected for being generable, so the last bin is not a like-for-like extension of the others. Nothing here rests on it.

THE TWO THIN CELLS FAIL IN OPPOSITE SHAPES

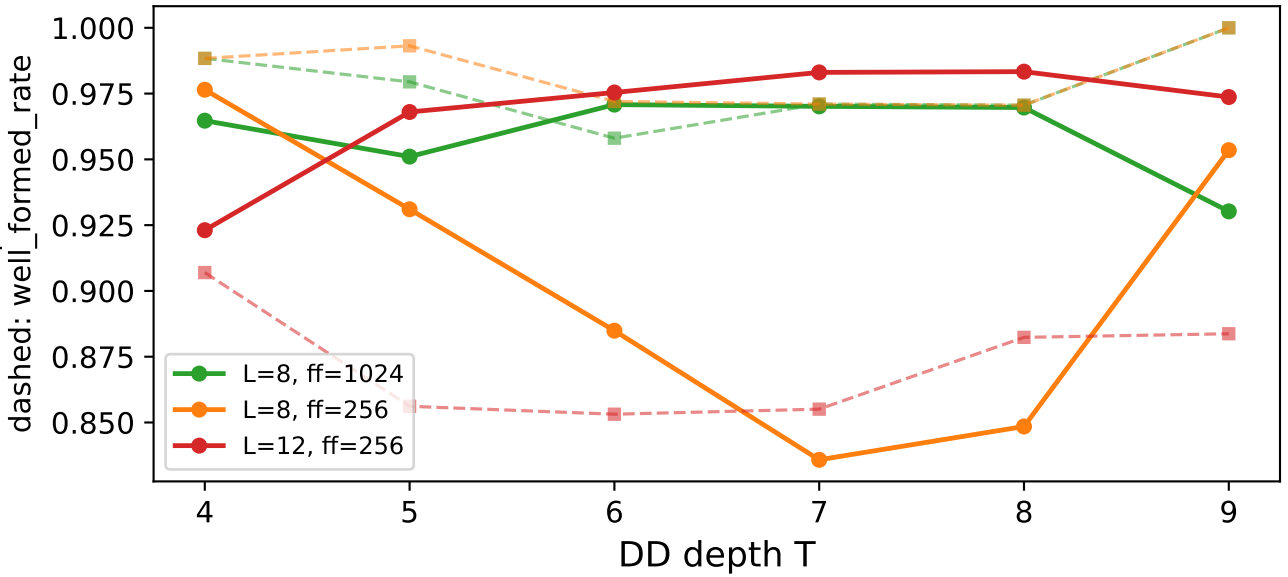
L=8/ff=256 (orange) declines with T the way a capacity-limited reasoner should, clearing the gate at T=4 and T=5 and falling to 0.812 by T=7.

L=12/ff=256 (red) is FLAT -- 0.829 to 0.868 across the whole range, and its Cochran-Armitage trend is positive, not negative. A model that is equally bad at T=4 and T=9 is not failing at deduction. The next page shows what it is failing at instead.

Cochran-Armitage trend over the six ordered bins, the same statistic findings 11 and 22 reported

The deep-thin cell stopped writing valid traces

well_formed_rate is the fraction of generations that parse as a trace at all.
Dividing solve_rate by it gives the solve rate CONDITIONAL on being well formed:
how good the deduction is, once the grammar is out of the way.



	worst well formed	solve|wf at T=4	solve|wf at T=9	r vs T
L=8, ff=1024	0.958	0.965	0.930	-0.39
L=8, ff=256	0.971	0.976	0.953	-0.38
L=12, ff=256	0.853	0.923	0.974	+0.72

WHAT THE TWO PANELS SAY

L=12/ff=256 (red): well-formedness collapses, but once it parses it solves 0.92-0.98 and IMPROVES with T -- deduction near the wide knee's, output malformed.

L=8/ff=256 (orange): the reverse. Well formed throughout, conditional rate declining 0.976 -> 0.836 -- the genuine depth-limited reasoner, and the cell that defines the width cost.

WHY THIS MATTERS FOR READING L*

L*(96, ff=256) > 12 is correct as stated -- L=12 does not clear the gate -- but it is NOT evidence that 12 thin layers cannot do depth-9 deduction. The pool agrees the failure is generative (truncated_rate 0.0007 -> 0.0167, outcome_accuracy 0.986 -> 0.869). At ff=256 a 12-layer stack becomes hard to TRAIN TO EMIT the format, and that confounds the knee.

well_formed_rate from sudoku_depth_eval; pool measures from sudoku_agg_eval

The shared pool, and a reference that reproduced

cell	solve_rate	Wilson 95%	delta	won/lost	McNemar p	trunc	outcome acc
L=8, ff=1024	0.9113	[0.8959, 0.9247]	reference	--	--	0.0000	0.9787
L=8, ff=256	0.8407	[0.8213, 0.8583]	-0.0707	74/180	2.3e-11	0.0007	0.9860
L=12, ff=256	0.8100	[0.7894, 0.8291]	-0.1013	76/228	8.4e-19	0.0167	0.8687

A FREE REPLICATION CHECK

The reference checkpoint L=8/ff=1024 was scored by TWO independent agg_eval calls a day apart -- once in job 49639312's pool and once here -- and both report 0.911333 on 1500 instances. Same seed_start (5e10), same INSTANCE_KEYS, so the pool is reconstructed identically and generation is deterministic. That is a check on the harness, not a result, but it is the kind of thing that is cheap to verify and expensive to discover later.

WHAT IS PAIRED HERE AND WHAT IS NOT

All three cells share the pool: they differ only in architecture, and shuffle_adj is False in all of them, so the prompts are byte-identical and McNemar applies. The L*(N, ff) comparisons ACROSS rows of the previous page's grid are not paired -- different N means a different instance distribution and a different vocabulary -- so the 'width factor' column is a comparison of two knees, each measured on its own ladder, and carries no p-value.

Pre-registrations, as resolved

Written into scripts/slurm/sudoku2_n96_ff256_lstar.sbatch before the job ran.

	claim	verdict	evidence
(a)	L=8/ff=256 does NOT clear all six rho=0.60 bins	CONFIRMED	clears 3 of 6 (T=4, 5 and 9). The width cost of finding 14 is present at N=96.
(b)	L=12/ff=256 DOES clear all six, i.e. L*(96, ff=256) <= 12	FALSIFIED	clears 0 of 6. The named falsifier -- 'a super-multiplicative width cost, which needs its own replication' -- holds.
(c)	L=12/ff=256 is at least as good as L=8/ff=1024 at 0.76x parameters	FALSIFIED	0.8100 vs 0.9113, -0.1013, p=8.4e-19. Resolvably worse.
(d)	Inside whichever thin column clears, L*(T) rises with T	UNRESOLVABLE	neither thin column clears, so the test has no subject. The L=12 trend is in fact POSITIVE (CA z=+0.67, p=0.503).

THREE OF FOUR WRONG, ALL IN THE SAME DIRECTION.

(b), (c) and the hypothesis in the header all assumed the N=48 width factor would transfer. It did not, and my pre-registered falsifier for (b) names exactly what happened, which is the only reason this reads as a result rather than a surprise. (d) is a design fault, not a prediction error: I made it conditional on a cell clearing, so a two-cell job that fails twice cannot answer it at all.

L=16 was deliberately excluded from the grid: it would have pushed training past the eval stage's start

What this changes, and what comes next

CHANGED

- Every L* in STATUS.md keeps its 'at ff=1024' caveat, and the caveat is now known to be load-bearing rather than cautious: at N=96 the thin knee is above 12 while the wide one is 8.
- Finding 13's 'depth is the cheaper currency' is demoted to small N. At N=96 the extra layers did not merely fail to pay -- they cost 3.1 points.
- The L* vs (N, ff) grid now has five of six entries, and the missing one (N=24 at ff=256) is the cheapest cell in the project to fill.

NOT CHANGED

- These are DECODING scores. shuffle_adj=False, so the adjacency rows still arrive sorted by the neighbour's true colour (finding 16). The whole L* ladder lives in the leaked task, and job 49705064 has only just established the budget a leak-free ladder would need.
- The L=12 result is confounded by trainability, not just capacity (previous page).
- One seed per cell, and L* is read off a 0.90 gate on six bins -- a coarse instrument that cannot distinguish L*=13 from L*=20.

NEXT, IN ORDER OF INFORMATION PER GPU-HOUR

1. Re-run L=12/ff=256 with a cosine schedule and warmup. If well_formed_rate recovers to 0.98 and the bins clear, the knee was trainability; if it stays flat, L*(96, ff=256) > 12 is a statement about computation after all.
2. L=10/ff=1024 at N=96, to tighten the wide knee from 'between 6 and 8' to one rung.
3. N=24 at ff=256 -- hours, not a day -- to get the width factor at the small end and see whether it grows with N monotonically.

checkpoints: d0256_h04_L08_ff00256_lr1e-03_pre and d0256_h04_L12_ff00256_lr1e-03_pre in sudoku_n96_ff256_2026-10-27